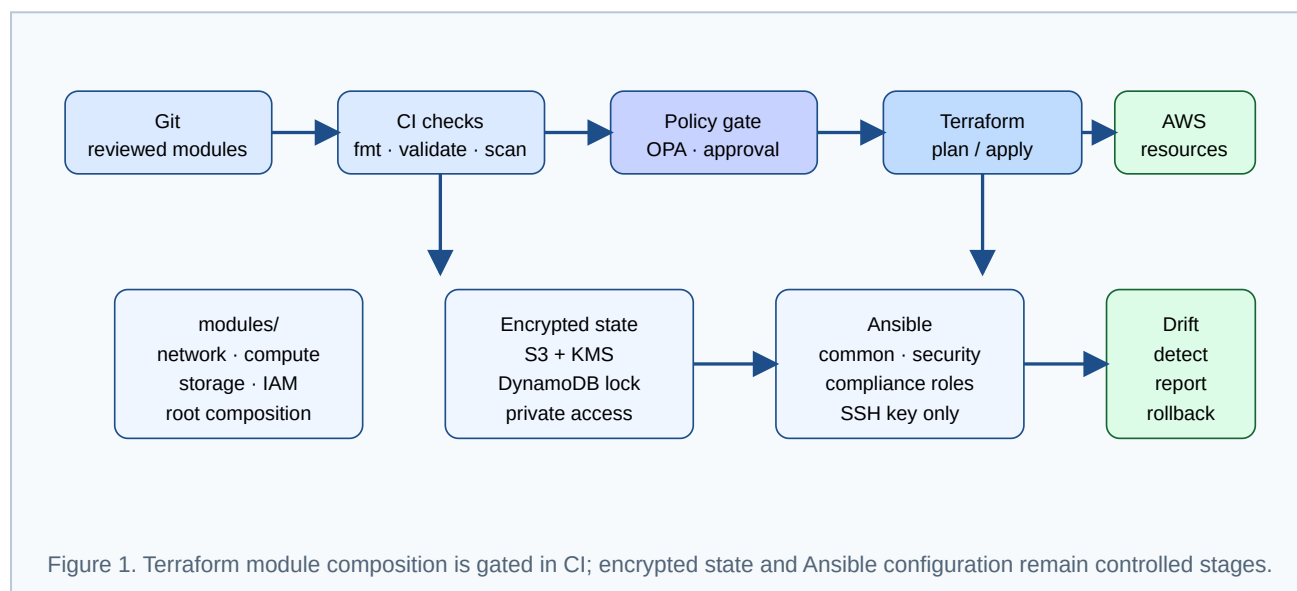


Task 4 — IaC Security Architecture

Secure Terraform + Ansible cloud deployment architecture

Security objective and separation of duties

The architecture treats infrastructure as a versioned, reviewable supply chain. Terraform provisions the cloud foundation and long-lived resources; Ansible configures operating-system state after an instance is reachable. The two tools share outputs through a controlled inventory artifact, not through copied secrets. Pull requests, plan review, policy checks, and an approval gate separate the person proposing a change from the person applying it. Every apply is tied to a commit, workspace, actor, and plan checksum so an auditor can reconstruct what was intended and what was deployed.



Terraform modules and state security

The root module composes small network, compute, storage, and identity modules with typed variables and explicit outputs. The VPC has private-by-default data paths, narrowly scoped security groups, and only the required ingress ports. The S3 bucket is private, versioned, encrypted with a customer-managed KMS key, and protected by public-access-block settings. The EC2 launch configuration requires a key pair, a current approved AMI supplied as an external variable, and bootstrap settings that disable root SSH and password authentication. Outputs expose identifiers and endpoints, never secret material.

Terraform state is more sensitive than the configuration because it can contain addresses, metadata, and provider-returned values. The backend therefore uses a dedicated private S3 bucket with versioning, server-side KMS encryption, restrictive bucket policy, TLS-only access, and a DynamoDB lock table. Backend credentials come from the CI identity or a short-lived role, not from `terraform.tfvars`. State access is separated from application access, CloudTrail records reads and writes, and recovery uses an earlier version after incident review. A local backend is permitted only for isolated syntax tests; it is never the production source of truth.

Secrets and policy enforcement

Secrets are kept in AWS Secrets Manager or an equivalent vault and injected only at runtime. They are never committed, placed in Terraform variables, printed in plans, or passed as Ansible command-line arguments. CI uses OIDC federation to assume a short-lived deployment role, with separate plan and apply permissions. Static checks run Terraform format and validate, tfsec or Checkov-style misconfiguration scans, dependency/provider lock verification, and secret scanning. Policy-as-code rejects public buckets, unrestricted SSH, unencrypted storage, missing tags, unapproved regions, and IAM wildcards. An approval is required for network exposure, IAM changes, and destructive actions.

Ansible configuration and compliance

After Terraform publishes a private inventory containing instance addresses, Ansible applies idempotent roles. The common role updates packages, sets the approved timezone, and configures logging. The security role enforces key-only SSH, disables root login, configures a deny-by-default firewall with only required ports, installs fail2ban, enables auditd, and applies unattended security updates. The compliance role checks file permissions, SSH settings, firewall state, audit service status, and package posture, then writes a passed/failed report. Handlers restart services only when templates change. Vault-encrypted variables or a secrets manager protect private values.

Drift, rollback, and validation

A scheduled, read-only Terraform plan detects cloud drift and sends a review alert; it does not silently overwrite manual changes. Ansible check mode and a regular compliance run detect operating-system drift. Remediation is approved through the same pipeline. Rollback uses a known-good Git commit and a reviewed Terraform plan, restores prior state versions only when necessary, and reapplies the matching Ansible role version. Before release, CI runs unit checks, policy validation, a plan, and a disposable environment test. After release, health checks, CloudTrail, AWS Config-style compliance rules, and the generated compliance report provide evidence. These controls align with HashiCorp Security Best Practices: protect state, use least privilege, separate environments, validate changes, and keep credentials out of source.

Environment promotion is deliberately boring: development, staging, and production use the same modules with different reviewed variables and separate state. A failed health check pauses promotion rather than triggering an automatic rollback with unknown consequences. The incident owner records the failed control, affected resources, and recovery decision. Backups of state and immutable build artifacts make recovery possible even if the source branch is damaged. Periodic access reviews remove stale deploy roles, and provider upgrades are tested in a disposable workspace before the lock file is changed.

Reference: HashiCorp, *Security Best Practices*, Terraform documentation at developer.hashicorp.com/terraform.

Prepared for the Week 06–07 Combined Assignment · AI, Automation & Security Engineering